

Superscalar

processors

- Scalar processors are those which execute one instruction per cycle and only one completion of instruction is expected from the pipeline per cycle.
- In a superscalar processor, multiple instructions are issued per cycle and multiple results are generated per cycle.
- Superscalar processors are designed to exploit more instruction-level parallelism in user programs.
- The amount of instruction level parallelism varies widely depending on the type of code being executed.
- A superscalar processor of degree m can issue m instructions per cycle.
- To fully utilize the superscalar processor of degree m , m instructions must be executed in parallel, which may not be possible in all clock cycles. In this situation some of the pipelines may be stalling in a wait state.

*refer book page 164-167 fig 4.12 and 4.13 (page no. may vary in different editions)

Superscalar Processor

- Superscalar Processors:

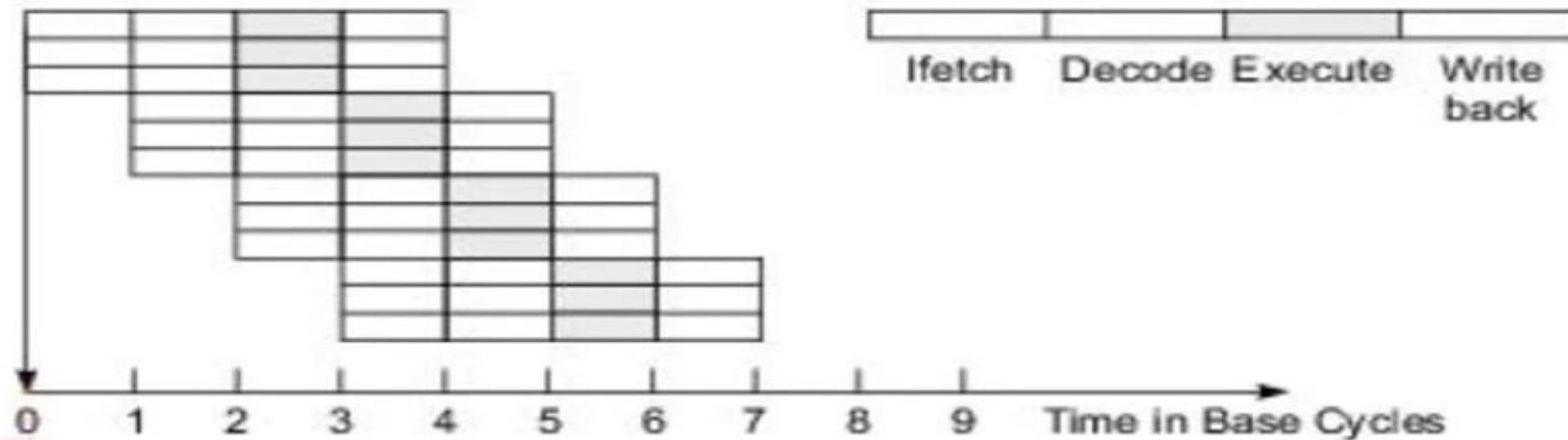
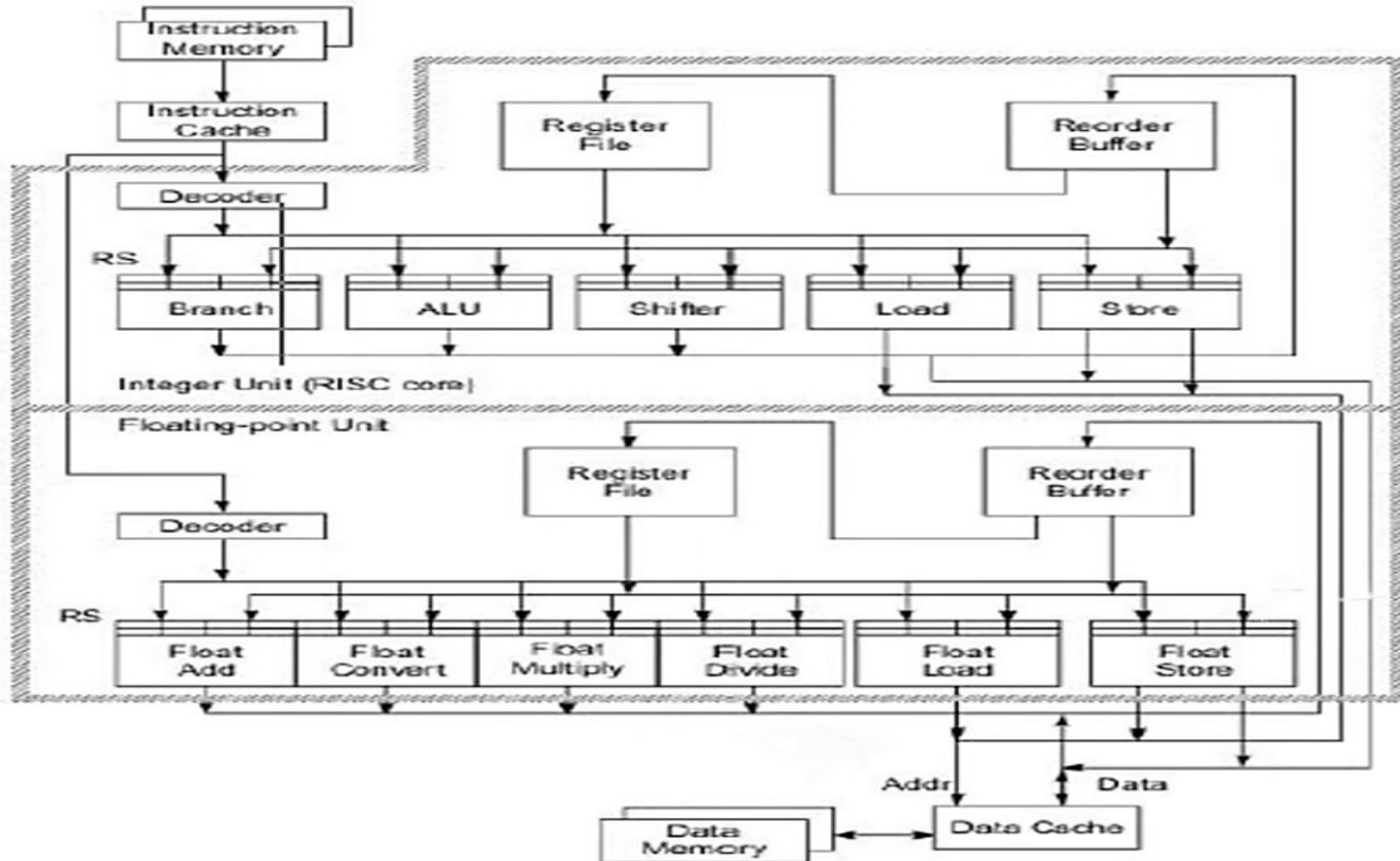


Fig. 4.11 A superscalar processor of degree $m = 3$

Superscalar RISC Processor

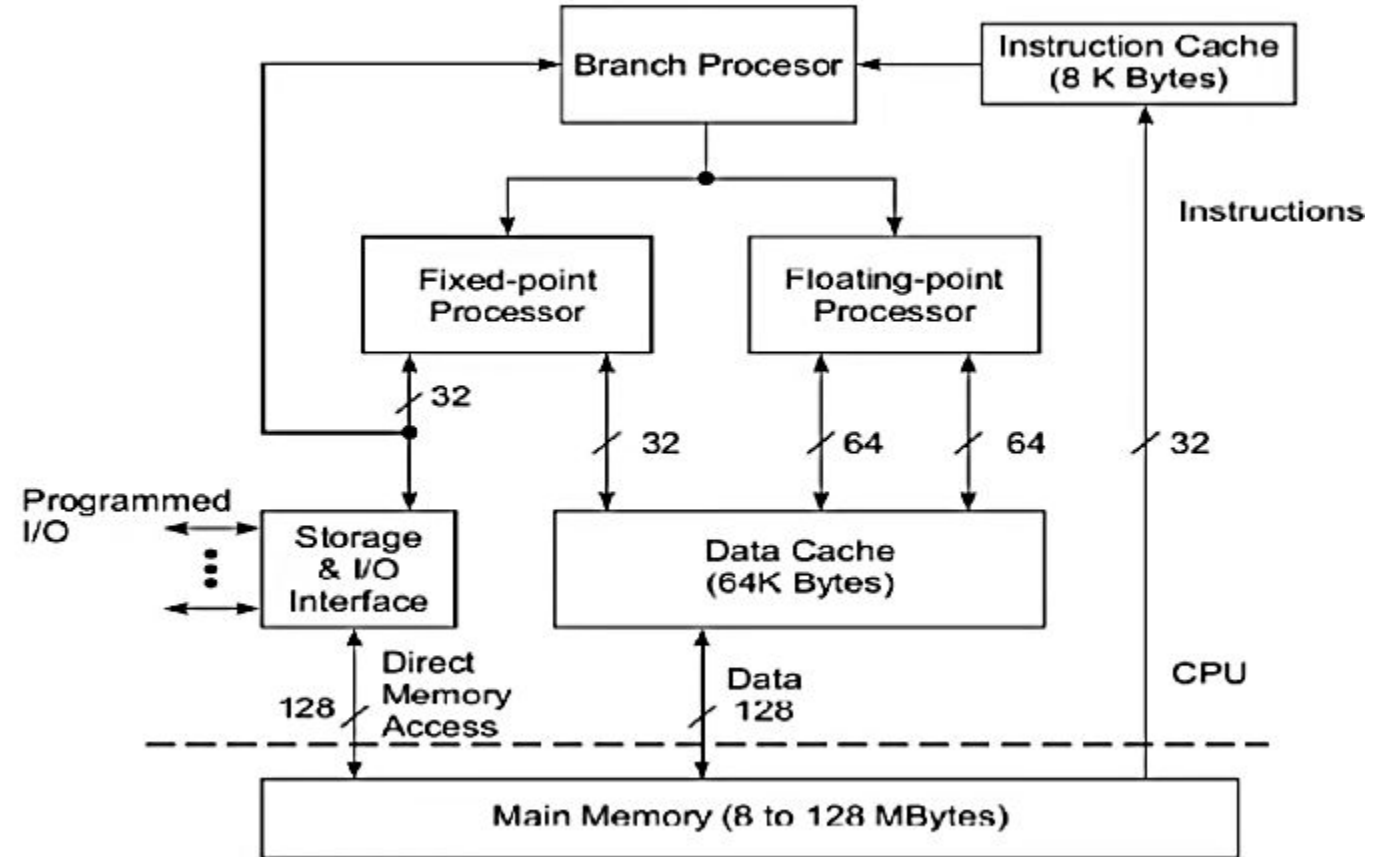


IBM RS/6000

Superscalar Processor

IBM RS/6000

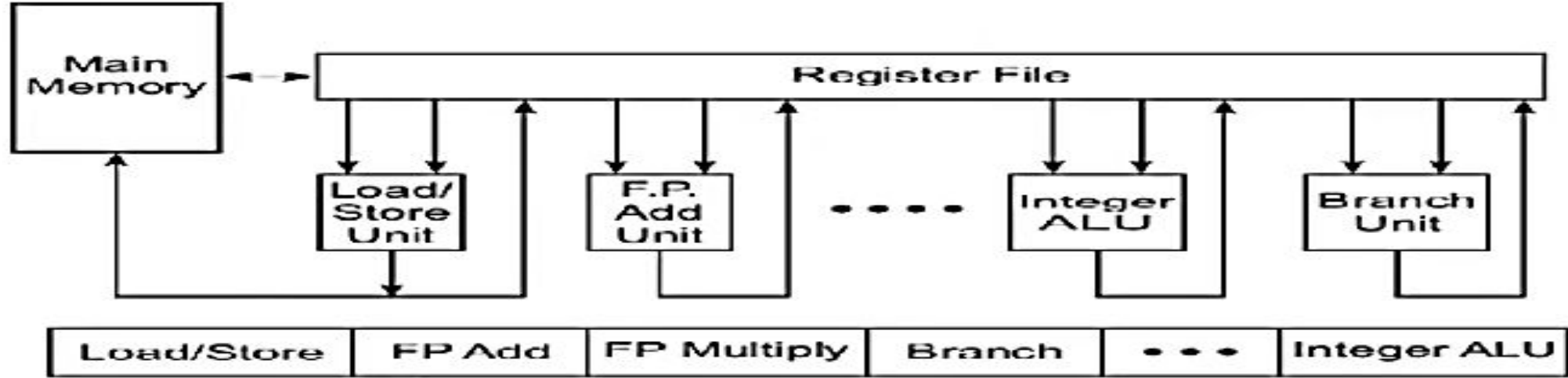
- 3 FUs
 - Fixed point unit
 - Floating point Unit
 - Branch processor unit - arranges execution of 5 IPC
- Hardwired control
- Wide buses



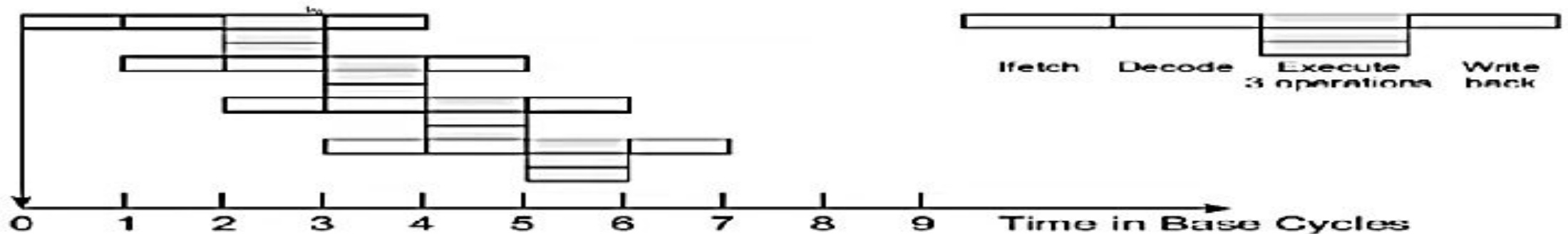
IBM RS/6000 Processor

- The architecture of this processor has its instruction set specifically designed for a superscalar machine containing three independent units i.e. branch, fixed-point, and floating-point.
- This design also emphasizes high-performance floating-point operations.
- The design principles are to offer maximum overlap of the three functional units, avoid dead cycles, and define instructions that can (for the most part) be completed at a rate of one per cycle.
- The branch processor can execute upto 5 instructions per cycle.
- The wide buses provide the high instruction and data bandwidths required for superscalar implementation.

VLIW (Very Long Instruction Word)



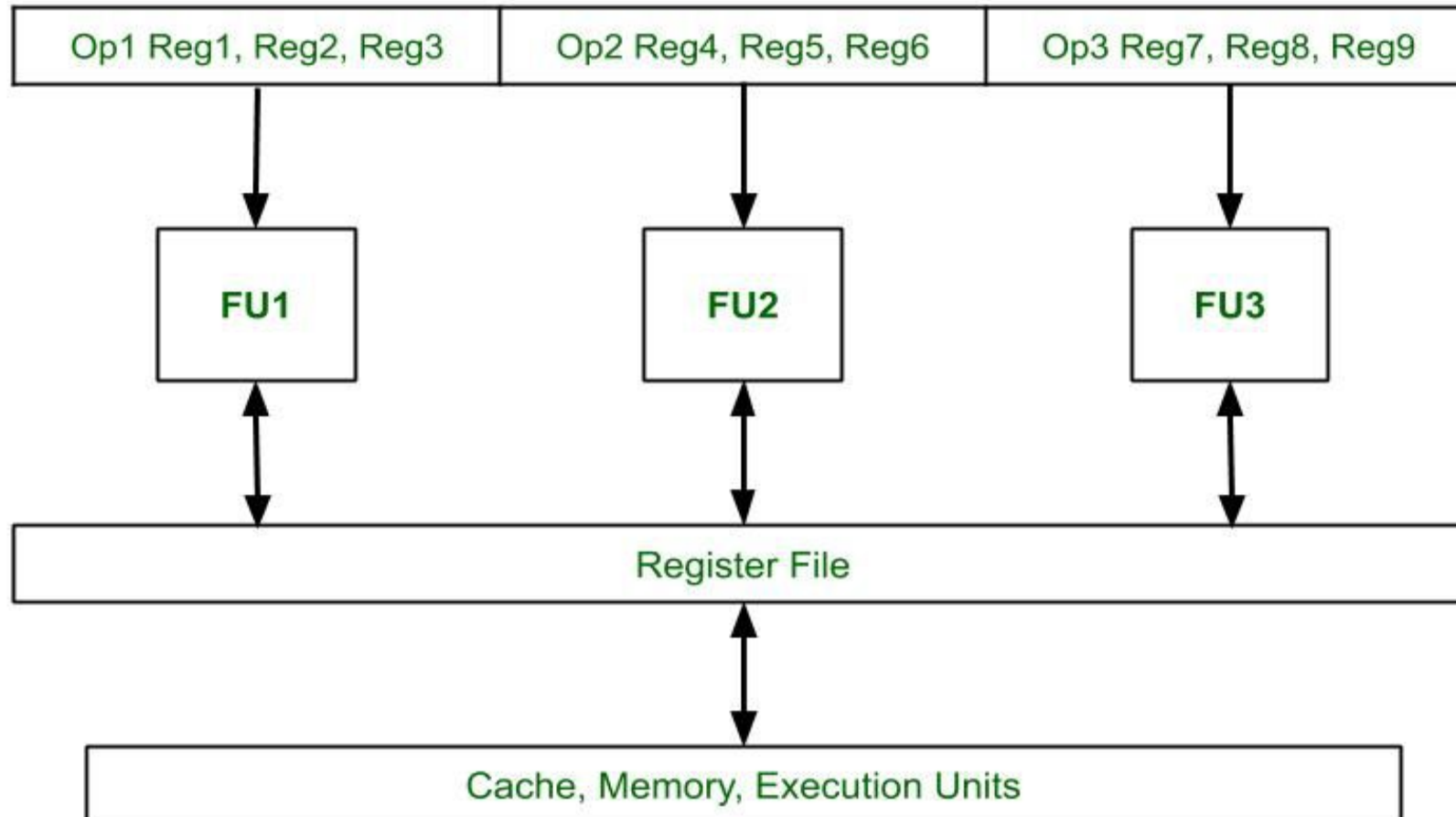
(a) A typical VLIW processor with degree $m = 3$



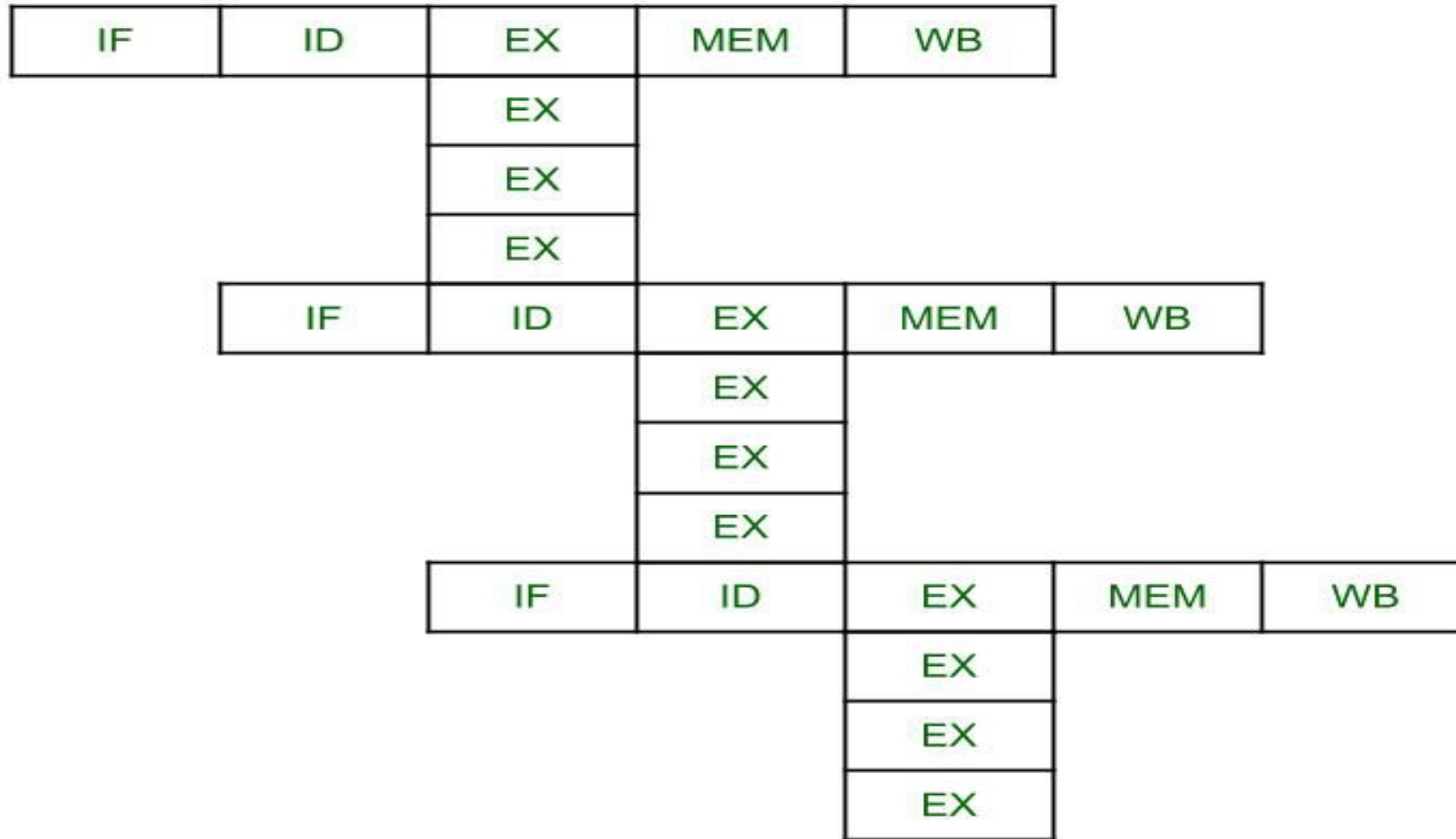
(b) VLIW execution with degree $m = 3$

- Very Long Instruction Word (VLIW) is a type of processor architecture designed to execute multiple operations in a single instruction cycle.
- VLIW architecture is generalized from two concepts: horizontal microcoding and superscalar processing.
- Unlike conventional processors that rely heavily on hardware to find parallelism, VLIW shifts this responsibility to the compiler, which packs independent instructions into one long instruction word.
- Execution units work simultaneously on different operations.
- Reduces the need for complex runtime scheduling.
- Improves instruction-level parallelism.

Block Diagram



Time space diagram of VLIW



IF: Instruction Fetch ID: Instruction Decode EX: Execute MEM: Memory WB: Write Back

Note: **In other architectures, the performance of the processor is improved by using either of the following methods: pipelining (break the instruction into subparts), superscalar processor (independently execute the instructions in different parts of the processor), out-of-order-execution (execute orders differently to the program) but each of these methods add to the complexity of the hardware very much. VLIW Architecture deals with it by depending on the compiler. The programs decide the parallel flow of the instructions and to resolve conflicts. This increases compiler complexity but decreases hardware complexity by a lot.

VLIW vs Superscalar

	VLIW	Superscalar
Decoding	Easier - Simple decoding	Complex
Code density	When ILP is lesser, code density worsens	Better code density when ILP is less
CPI (Cycles per instruction)	Lower	Better
Effectiveness	Depends on efficiency of code compaction	Dynamic (adaptive) behavior
Detecting parallelism	Requires explicit encoding of parallelism. However, No need for additional H/w or S/w to detect parallelism. [Simpler hardware design]	Complex hardware design. Requires hardware and software (compiler) support.

Vector

Processors

- A vector processor executes vector instructions on arrays of data, each vector instruction involves a string of repeated operations (a vector instruction involves a large array of operands), which are ideal for pipelining with one result per cycle.
- A vector processor can be assumed a register-to-register architecture or a memory-to-memory architecture.
- Register-to-register architecture uses shorter instructions and vector register files.
- The memory-to-memory architecture uses memory based instructions which are longer in length including memory addresses.

**A vector instruction is a type of computer instruction that allows a processor to perform the same operation on multiple data elements simultaneously, essentially processing an entire array of data with a single instruction, leading to significantly improved performance for applications that involve large data sets like image processing, scientific simulations, and video encoding.

Vector Processors

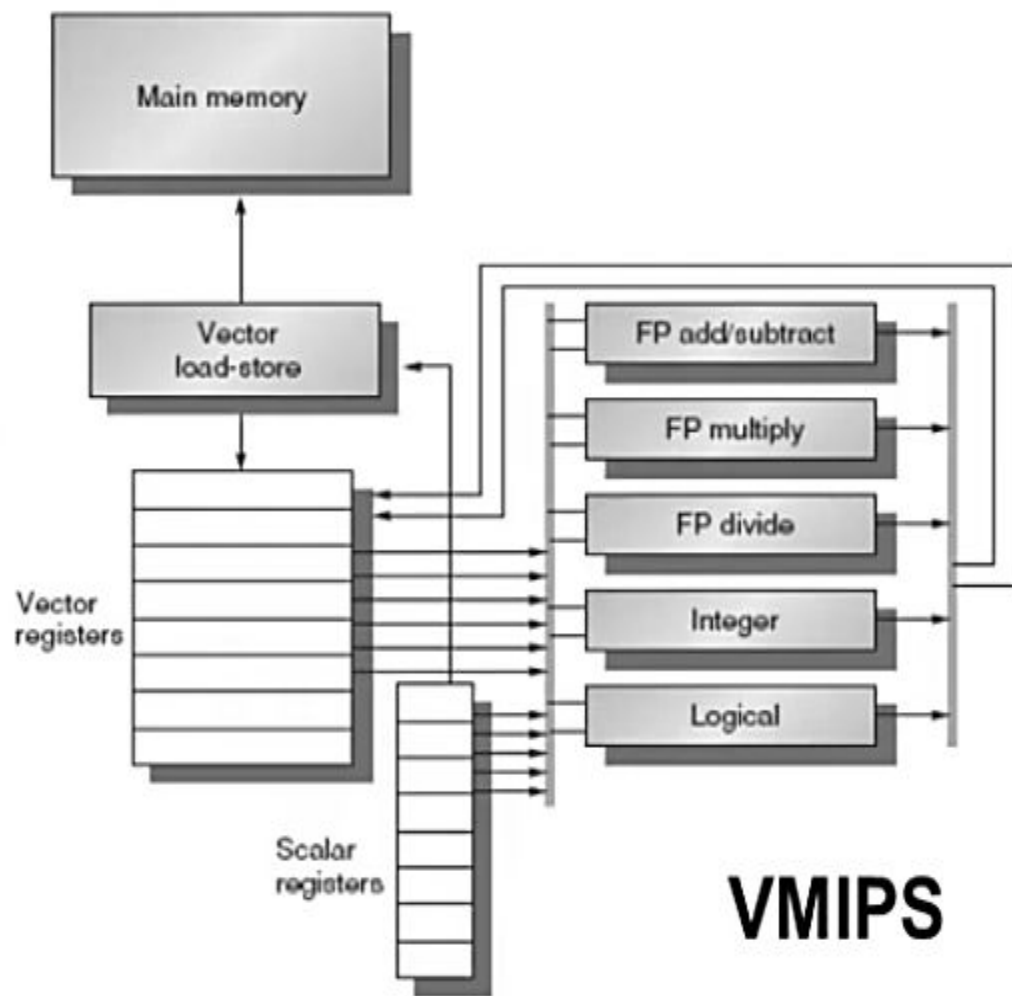
Memory to memory VP

- Memory based instructions
- Longer instructions
- Instructions include memory address

Register to register VP

- Shorter instructions
- Vector register files

The basic structure of a vector-register architecture



Vector Processors

Vector Instructions (Register to register)

- Binary Vector $\mathbf{V1} \circ \mathbf{V2} \rightarrow \mathbf{V3}$
 - Scaling $\mathbf{sI} \circ \mathbf{V1} \rightarrow \mathbf{V2}$
 - Binary Reduction $\mathbf{V1} \circ \mathbf{V2} \rightarrow \mathbf{sI}$
 - Vector Load $\mathbf{M(I:n)} \rightarrow \mathbf{V1}$
 - Vector Store $\mathbf{V1} \rightarrow \mathbf{M(I:n)}$
 - Unary Vector $\circ \mathbf{V1} \rightarrow \mathbf{V2}$
 - Unary Reduction $\circ \mathbf{V1} \rightarrow \mathbf{sI}$
-

Vector Processors

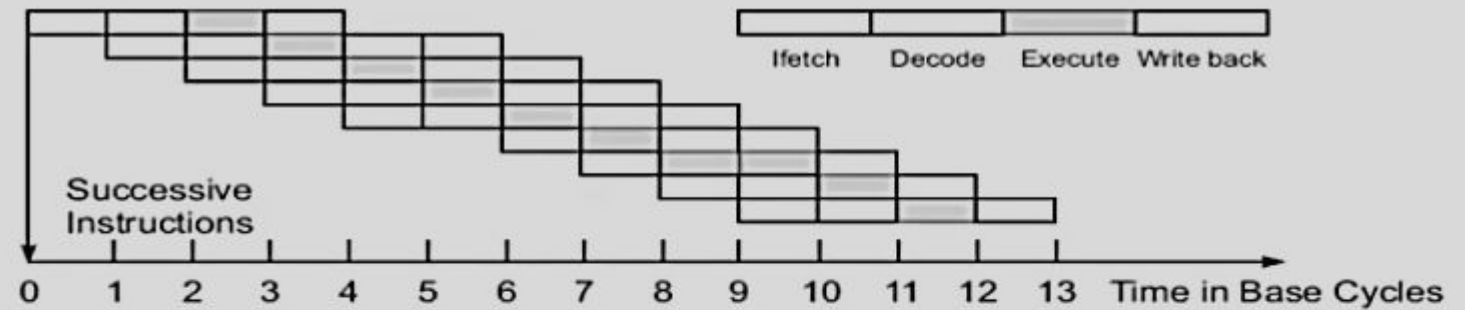
Vector Instructions (Memory to memory)

$M_1(l:n) \circ M_2(l:n) \rightarrow M_3(l:n)$

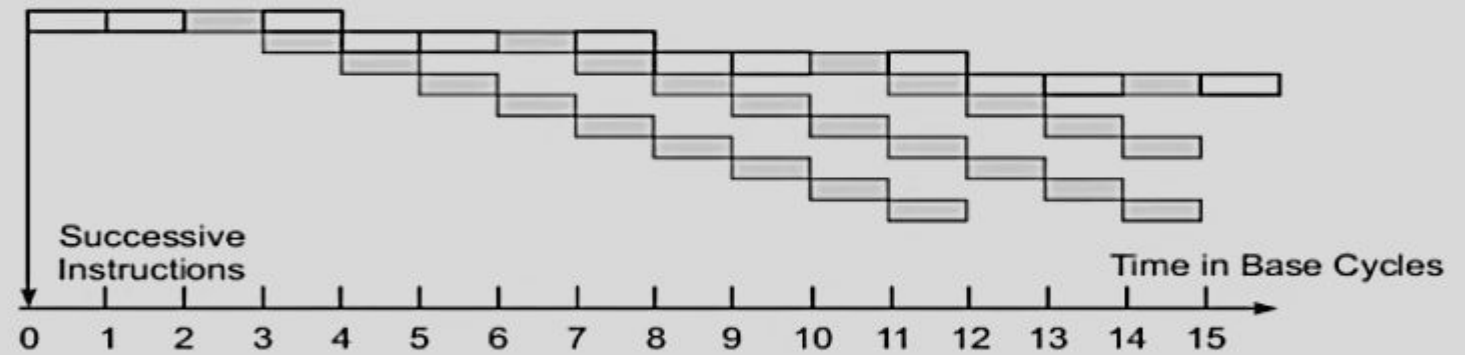
$s_1 \circ M_1(l:n) \rightarrow M_2(l:n)$

$\circ M_2(l:n) \rightarrow M(l:n)$

$M_1(l:n) \circ M_2(l:n) \rightarrow M(k)$



(a) Scalar pipeline execution (Fig. 4.2a redrawn)



(b) Vector pipeline execution

Vector Pipelines

Attached to any scalar or superscalar processor

- **Principles of vector processing**
- **Vector instructions**
- Vector instructions perform parallel operations on multiple data elements simultaneously. They are also known as single-instruction multiple-data (SIMD) instructions.
- **Vector arrays**
- A vector is an ordered set of data items in a one-dimensional array.
- **Vector processing in a pipeline**
- Data processing elements are connected in series, with the output of one being the input of the next.
- **Vector processing in an array**
- One process is broken into multiple sub-processes, and each sub-process is processed by a separate array.